

Tunisian Republic
Ministry of Higher Education
and Scientific Research

University of Monastir
Faculty of Sciences of
Monastir



Bachelor's Degree in:

Computer Science

Option :

Software Engineering
and Information Systems

Ordre N° : *****

Thesis

Presented to

The Faculty of Sciences of Monastir

for obtaining the

Bachelor's Degree in Computer Science

Option :

Software Engineering
and Information Systems

by

Hamza Znaidi

**Design and development of a web application for
production tracking and stock management**

Presented on: [Date], before the jury composed of:

Dedication

To my dear parents

For all your sacrifices, your unconditional love, your unwavering support, and your prayers that have accompanied me throughout my studies.

To my dear family

Thank you for your constant encouragement and precious moral support.

To my brothers, sister, and my closest friends

Thank you for always being there for me and supporting me in difficult times.

I hope this work will meet your expectations and reflect your unwavering support. I am infinitely grateful to have you in my life.

*To all of you,
I dedicate this modest work.*

Acknowledgments

Before addressing the content of this work, I would like to express my deep gratitude to all those who contributed, directly or indirectly, to the completion of this end-of-studies project.

I would particularly like to thank:

- **Mr. Souheyl Mallat** - For academic guidance and valuable feedback throughout the project
- **Mr. Mehrez Ayadi** - For professional mentorship and technical expertise
- **The KTS Team** - For their cooperation and for providing insights
- **The Department of Computer Science** - For providing the educational foundation necessary for this work
- **My colleagues** - For their support, collaboration, and constructive discussions

I do not forget all those who, through their discreet but constant encouragement, enabled me to carry out this work successfully. May they find here the expression of my gratitude.

Finally, I dedicate these acknowledgments to all the teachers who trained me throughout my university career.

Contents

Dedication	i
Acknowledgments	ii
List of Abbreviations	ix
General Introduction	1
1 General context and preliminary study	2
1.1 Introduction	2
1.2 Project Framework	2
1.3 Presentation of the Host Organization	2
1.4 Study of Existing Processes	3
1.4.1 Description of Current State	3
1.4.2 Critique of Existing System	3
1.4.3 Existing Solutions	4
1.4.3.1 SMOFT ERP	4
1.4.3.2 SAP Digital Manufacturing	5
1.4.4 Proposed Solution	6
1.5 Work Methodology	8
1.5.1 Agile Scrum Method	8
1.6 Methodology: Scrum Sprint Summary	8
2 Analysis and Specification of requirements	10
2.1 Introduction	10
2.2 Specification of requirements	10
2.2.1 actors identification	10
2.2.2 Functional Specifications	10
2.2.2.1 User and Authentication Management	10
2.2.2.2 Stock Management	11
2.2.2.3 Work Order Management	11
2.2.2.4 Chat and Communication	11
2.2.2.5 Notification System	11

2.2.2.6	Production Stages	11
2.2.3	Non-Functional Specifications	12
2.2.3.1	Performance Requirements	12
2.2.3.2	Scalability Requirements	12
2.2.3.3	Security Requirements	12
2.2.3.4	Usability Requirements	12
2.2.3.5	Compatibility Requirements	12
2.2.4	Technical Requirements Specification	13
2.3	Global use case diagram	15
2.4	Global class diagram	15
2.5	Planning	17
2.5.1	Product backlog	17
2.5.2	Sprint Planning	18
2.6	Conclusion	19
3	Release 1	20
3.1	Introduction	20
3.2	Sprint 1: Foundation and Access Control	20
3.2.1	Sprint 1 Backlog	20
3.2.2	Sprint 1 use case diagram	21
3.2.3	User Login and Protected Resource Access — JWT Sequence Diagram	21
3.2.4	Sprint 1: Application Interface Overview	22
3.3	Sprint 2: Work Orders and Stage Tracking	23
3.3.1	Sprint Backlog	23
3.3.2	Sprint 2 use case diagram	24
3.3.3	Sequence Diagram for Work Order Progress Update	24
3.3.4	Sprint 2: Application Interface Overview	26
3.4	Sprint 3: Stock Management	26
3.4.1	Sprint Backlog	26
3.4.2	Sprint 3 use case diagram	27
3.4.3	Sprint 3: Application Interface Overview	27
3.5	Conclusion	28
4	Release 2	29
4.1	Introduction	29
4.2	Sprint 4: Notes, Real-Time Chat, and Notifications	29
4.2.1	Sprint Backlog	29
4.2.2	Sprint 4 Use Case Diagram	30
4.2.3	Sequence Diagram for Real-Time Messaging	30

4.2.4	Sprint 4: Application Interface Overview	32
4.3	Sprint 5: QR Codes, Analytics, and AI Reporting	33
4.3.1	Sprint Backlog	33
4.3.2	Sprint 5 Use Case Diagram	33
4.3.3	Sequence Diagram for AI-Based Report Generation	34
4.3.4	Sprint 5: Application Interface Overview	34
4.4	Conclusion	35
	General Conclusion	36
	References	37

List of Figures

1.1	Host organisation KTS logo [3]	2
1.2	SMOFT official logo [5]	4
1.3	SMOFT dashboard	5
1.4	SAP Digital Manufacturing [7]	5
1.5	SAP Digital Manufacturing dashboard [8]	6
1.6	SCRUM Process [10]	8
2.1	Use Case Diagram	15
2.2	Global class diagram	16
3.1	Sprint 1 Use Case Diagram	21
3.2	Authentication Sequence Diagram	22
3.3	Login Interface	23
3.4	User Management Interface	23
3.5	User Creation Modal	23
3.6	Sprint 2 Use Case Diagram	24
3.7	Work Order Progress Update Sequence Diagram	25
3.8	Work Order Management Interface	26
3.9	Production Stages Admin View	26
3.10	Worker Tasks View	26
3.11	Sprint 3 Use Case Diagram	27
3.12	Inventory Admin View	28
3.13	Inventory Reservations Admin View	28
3.14	Inventory Worker View	28
3.15	Inventory Reservations Worker View	28
4.1	Sprint 4 Use Case Diagram	30
4.2	Real-Time Chat Message Flow Sequence Diagram	31
4.3	Note Management Worker View	32
4.4	Notes History Admin View	32
4.5	Chat Messaging Interface Worker View	32
4.6	Chat Messaging Interface Admin View	32
4.7	Notification Center	32

4.8 Sprint 5 Use Case Diagram 33

4.9 Sequence Diagram for AI-Based Report Generation 34

4.10 Work Order Details Interface with QR Code 35

4.11 QR Code Scanner Interface 35

4.12 AI Report Interface 35

4.13 Analytics Dashboard 35

List of Tables

1.1	General Information about KTS	3
1.2	Comparison of Existing Solutions with Our Project	7
1.3	Task by Scrum Sprint	9
2.1	Software environment technologies used	13
2.2	Product backlog	17
2.3	Sprint Plan	19
3.1	Sprint 1 Backlog	20
3.2	Sprint 2 Backlog	23
3.3	Sprint 3 Backlog	27
4.1	Sprint 4 Backlog	29
4.2	Sprint 5 Backlog	33

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ERP	Enterprise Resource Planning
FSM	Faculty of Sciences of Monastir
IT	Information Technology
JWT	JSON Web Token
KPI	Key Performance Indicator
KTS	Kabelbaum Test Systems
MES	Manufacturing Execution System
ORM	Object-Relational Mapping
PFE	End-of-Studies Project (Projet de Fin d'Etudes)
QR	Quick Response
RAM	Random Access Memory
REST	Representational State Transfer
SCRUM	Agile project management framework
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface
VS Code	Visual Studio Code

General Introduction

Manufacturing execution systems (MES) are computerized systems used in manufacturing to track and document the transformation of raw materials to finished goods. MES provides information that helps manufacturing decision-makers understand how current conditions on the plant floor can be optimized to improve production output[1].

Modern small-to-medium manufacturing workshops often face challenges such as delayed communication between planners and operators, manual paper-based status updates, inventory inconsistencies, and limited traceability of work history. To address these issues this project was carried out through a collaboration between the Faculty of Science of Monastir (FSM), and Kabelbaum Test Systems (KTS), the project focused on a scoped MES-style solution that implements work-order lifecycle management, stage tracking, basic inventory control, real-time messaging, and an alert/notification subsystem, along with analytical and reporting tools for production monitoring and performance evaluation.

This report presents the design and implementation of a production-tracking platform developed as part of an end of study project.

Report Structure

- **Chapter 1 – General context and preliminary study:** We will start by introducing the host company, along with the overall context of the project and its main issue. Next, we will examine and evaluate the current situation in order to identify its limitations and propose an effective solution. Finally, we will outline the methodology that will be followed to carry out this project.
- **Chapter 2 – Analysis and Specifications of requirements:** This chapter focuses on defining both functional and non-functional requirements, as well as the project's architecture. It concludes with the presentation of the overall use case and class diagrams, which serve as a foundation for developing the complete project backlog.
- **Chapter 3 and 4 – Release 1 and 2:** The following two chapters form the core of our report. These two chapters will be dedicated to the development of the two releases of our platform
- **General Conclusion:** Finally, we conclude with a general summary that presents a recap of the work completed and outlines some future directions.

Chapter 1

General context and preliminary study

1.1 Introduction

This chapter will explain where our project fits. We begin with the Project framework, presenting the host company, KTS (Kabelbaum Test Systems). Next, we make a study of the existing Process to address the problem that caused the host organization to develop this platform. We evaluate existing solutions to identify where they lack the company's needs and propose the direction of our solution. Finally, we present the methodology used for the development of our project.

1.2 Project Framework

This project was carried out as part of a final year project submitted to the Faculty of Sciences of Monastir for the award of a Bachelor's degree in Software Engineering and Information Systems. I worked at the company "KTS" for three months to design and develop a production tracking platform that helps the company to manage its production flow and inventory more efficiently.

1.3 Presentation of the Host Organization

KTS Kabelbaum Test Systems is a company established in 2019 in Bursa, Turkey, operating within a 2,000 m² indoor facility. The company was founded with the objective of meeting the needs of wiring harness manufacturers by providing specialized solutions and acting as a reliable solution partner in the industry[2].



Figure 1.1: Host organisation KTS logo [3]

KTS expanded to Tunisia in 2025 , all general information about the company is summerised in the table below:

Company Name	KTS (Kabelbaum Test Systems)
Founded	2025
Location	1 ZI Mghira, 65 Rue Khlidia, Ben Arous, Tunisia, 2082
Industry	Industrial Automation, Manufacturing Solutions
Phone number	+216 57 030 790
Website	https://www.ktsistem.com
E-mail	info-tn@ktsistem.tn
Director	Mehrez Ayadi

Table 1.1: General Information about KTS

1.4 Study of Existing Processes

1.4.1 Description of Current State

Production tracking at KTS currently operates through a paper-based workflow. where Work orders are created by the Sales department and circulate physically through all eight production stages. At each stage, workers manually update progress, documente completion of tasks, resource allocation, and readiness for the next phase. Inventory management is similarly manual where workers check physical stock records and update them by hand when materials are reserved or consumed. Communication between stages relies on direct contact and physical documentation, with no centralized system to track production progress or resource availability.

1.4.2 Critique of Existing System

The current paper-based system shows several limitations:

- **Lack of Traceability:** Information is divided across multiple physical documents, making it difficult to track the complete history of a work order's progression.
- **Communication Delays:** Physical communication between stages introduces delays and creates bottlenecks in the production workflow.
- **Visibility limitations:** Management has limited visibility into production status, and inventory levels.

- **Error Prone:** Manual data entry and documentation increases the risk of errors.
- **Scalability Issues:** As production volume increases, the manual system becomes increasingly difficult to manage.

the paper-based system is easy to use and has a low cost, but unfortunately, these aspects no longer satisfies the company's purposes.

1.4.3 Existing Solutions

1.4.3.1 SMOFT ERP

SMOFT ERP is a cloud-based enterprise resource planning platform that provides a modular set of business applications for managing operations across different departments. It focuses on centralizing information, supporting configurable workflows, and giving companies a unified environment for data management and process coordination[4].

- **Integrated business management:** Brings several operational functions into one platform.
- **Modular structure:** Can be adapted according to company needs and growth.
- **Centralized data:** Reduces information fragmentation across departments.

Weak points:

- It is mainly ERP-oriented, so it is not specialized for detailed shop-floor execution.
- Production-stage visibility and operator-level tracking are not the main focus.
- It can be broader than what a mid-sized manufacturing company needs for a dedicated production tracking workflow.



Figure 1.2: SMOFT official logo [5]

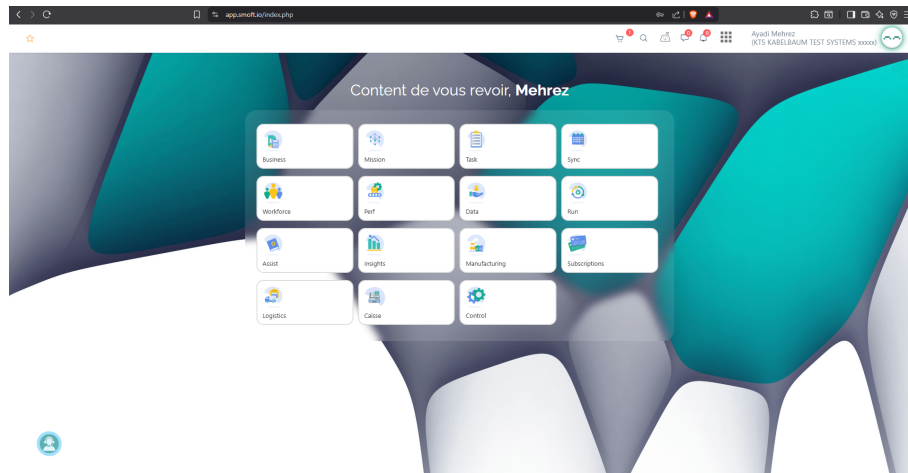


Figure 1.3: SMOFT dashboard

1.4.3.2 SAP Digital Manufacturing

SAP Digital Manufacturing is SAP's manufacturing execution and operations management solution. It connects shop-floor activities with enterprise planning, supports real-time production visibility, and helps manufacturing teams coordinate execution, traceability, and performance monitoring[6].

- **Shop-floor coordination:** Supports execution of production activities directly from the manufacturing floor.
- **Real-time visibility:** Gives managers current information about ongoing production.
- **Traceability support:** Helps track production progress and operational events.

Weak points:

- Deployment and integration can be complex for a small or medium company.
- The solution may require significant configuration and user training.
- It can be more expensive and heavier than a focused platform built for KTS-specific needs.



Figure 1.4: SAP Digital Manufacturing [7]

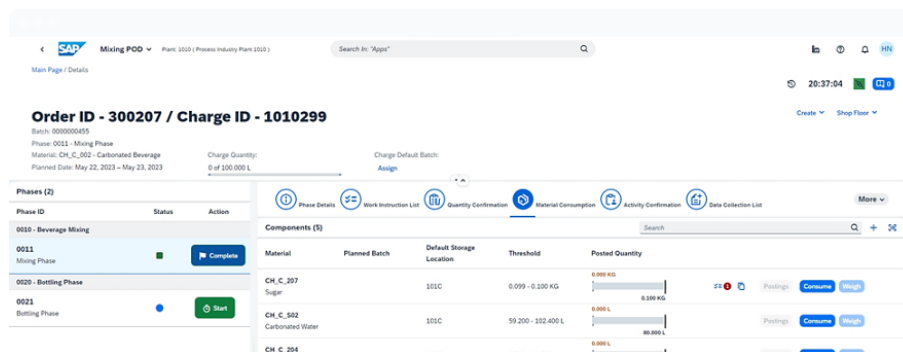


Figure 1.5: SAP Digital Manufacturing dashboard [8]

1.4.4 Proposed Solution

Two alternatives were considered:

Alternative 1: digital logbook system

Deploy a simple digital logbook system to replace paper forms while maintaining existing workflows. This approach minimizes disorder but can't improve processes that much.

Alternative 2: Production Management Platform

Develop a web-based application that standardizes workflows, offers role-based access control, provides real-time production visibility, and integrates inventory management.

The chosen approach is Alternative 2, a comprehensive platform, because it addresses not only the paper based process but also provide other solutions for better production management. The implementation includes:

- **centralized Tracking:** A unified platform where all work orders and tasks are digitally recorded.
- **Real-Time Visibility:** Administrators and workers access current status immediately, reducing communication delays.
- **Role-Based access:** Tailored interfaces and permissions for administrators, production workers.
- **Integrated inventory:** Stock management features linked directly to work orders, with reservation updates.
- **Chat messaging:** communication tool for better visibility and understanding.

- **Scalability:** A web-based architecture that grows with the company's production volume without the increase of manual efforts.

This solution is the best approach because it solves a lot of limitations and helps the company to grow efficiently.

The table 1.2 summarizes the comparison between the existing solutions and our project:

Table 1.2: Comparison of Existing Solutions with Our Project

Criterion	SMOFT ERP	SAP Digital Manufacturing	Our project
Traceability	Basic transactional traceability across business modules.	Strong production traceability for shop-floor operations.	Full traceability from work orders to stages, stock, notes, and chat.
Real-time visibility	Limited to ERP-level monitoring.	Good real-time production visibility.	Real-time visibility across production stages and inventory.
Role-based access	Available at business level, but not tailored to production roles.	Supports structured access for manufacturing users.	Dedicated access for administrators and production workers.
Usability for KTS	Too general for a dedicated production tracking workflow.	Powerful but heavier than required.	Designed specifically for KTS production tracking needs.
Deployment effort	Moderate, depending on customization.	High integration and configuration effort.	Lower effort because it is focused on the exact workflow.
Fit for KTS	Partial fit.	Partial fit, but more complex than necessary.	Best fit because it matches the company's process and constraints.

1.5 Work Methodology

1.5.1 Agile Scrum Method

[9] Scrum is a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems.

In a nutshell, Scrum requires a Scrum Master to foster an environment where:

- **1-** A Product Owner orders the work for a complex problem into a Product Backlog
- **2-** The Scrum Team turns a selection of the work into an Increment of value during a Sprint.
- **3-** The Scrum Team and its stakeholders inspect the results and adjust for the next Sprint.
- **4-** Repeat

During this project, **Mr. Ayadi Mahrez** served as the Product Owner and Scrum Master to ensure that there was supervision in the project, prioritization of requirements, and management of the Scrum process. The members of the Scrum Team comprised only me as the developer of the application

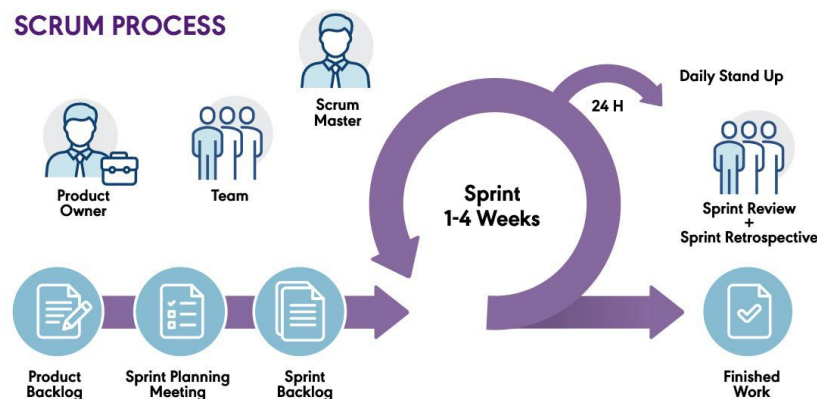


Figure 1.6: SCRUM Process [10]

1.6 Methodology: Scrum Sprint Summary

Five sprints were carried out using the Scrum methodology to implement a usable increment of the production Tracker platform in each step. The table 1.3 summarizes the objectives and goals of each sprint.

Table 1.3: Task by Scrum Sprint

Sprint	Objective	Goals
1	Foundation and Access Control	Establish authentication, authorization, and a stable user management foundation
2	Work Orders and Stage Tracking	Enable the core production workflow from work order creation to stage history.
3	Stock Management	Deliver a usable stock management system with validation and reliable inventory rules along with reservation functionality.
4	Notes, Real-Time Chat, and Notifications	Improve traceability and keep users informed in real time using communication and notification tools
5	QR Codes, Analytics, and AI Reporting	Add communication, quick access, and managerial visibility

Conclusion

the overall project environment has been described in this chapter. Where we presented the context of our project, starting with an introduction to the host organization and its activities, then we studied the existing process and solutions, which led us to identify limitations and weaknesses of the current system and propose the best solution, finally we presented the work methodology where we did an even distribution of roles and priorities, next chapter will be about the analysis and the specification of requirements.

Chapter 2

Analysis and Specification of requirements

2.1 Introduction

this chapter will present the analysis and specification of requirements used to develop this project, first we will identify the actors along with the functional and non-functional specifications, then we will present the technical requirements including the software environment , finally we will present the global use case diagram, class diagram and the product backlog.

2.2 Specification of requirements

2.2.1 actors identification

the main actors of the system are:

- **Administrators:** Responsible for managing the production process, assigning tasks, and overseeing operations.
- **Workers:** Execute assigned tasks, update work order statuses, and communicate with administrators.

2.2.2 Functional Specifications

2.2.2.1 User and Authentication Management

The System shall:

- provide secure user authentication with username and password.
- implement role-based access control (Admin, Worker).
- support user session management with token-based authentication.
- allow administrators to manage user accounts.

2.2.2.2 Stock Management

The System shall:

- keep record of inventory items with unique identifiers.
- maintain stock levels and provide low-inventory alerts.
- record stock movement history and reservations.

2.2.2.3 Work Order Management

The System shall:

- allow creation and assignment of work orders to workers.
- track work order status through different stages.
- generate QR codes for work orders for easy search.

2.2.2.4 Chat and Communication

The System shall:

- provide real-time messaging between users.
- support file attachments in chat messages
- maintain message history and conversation threads
- implement chat notifications for new messages

2.2.2.5 Notification System

The System shall:

- notify users of work order assignments in real-time
- alert users to inventory status changes
- provide notifications for chat messages .

2.2.2.6 Production Stages

The System shall:

- define production stages and workflows
- track stage progression and duration
- assign workers to specific production stages

2.2.3 Non-Functional Specifications

2.2.3.1 Performance Requirements

- System shall respond to user requests within 2 seconds under normal load
- System shall support concurrent users up to 500 without performance degradation
- Real-time notifications shall be delivered within 1 second of triggering event
- Page load time shall not exceed 3 seconds for desktop and 4 seconds for mobile

2.2.3.2 Scalability Requirements

- System architecture shall support horizontal scaling for increased load
- Database shall efficiently handle queries with up to 10 million records
- System shall support adding new features without major restructuring

2.2.3.3 Security Requirements

- All user passwords shall be hashed using industry-standard algorithms
- User data shall be protected with proper access control mechanisms
- API endpoints shall require proper authentication and authorization

2.2.3.4 Usability Requirements

- User interface shall follow modern UX/UI principles with consistent design
- System shall be accessible on desktop and mobile devices
- Navigation shall be intuitive requiring minimal training
- System shall provide clear error messages and help documentation

2.2.3.5 Compatibility Requirements

- System shall be compatible with modern web browsers (Chrome, Firefox, Safari, Edge)
- System shall work on iOS and Android mobile devices
- Backend APIs shall be independent of frontend implementation

2.2.4 Technical Requirements Specification

The software environment was organized around the technologies used to build, run, and maintain the platform. Each tool is presented by its logo and description below in the table 2.1.

Table 2.1: Software environment technologies used

Logo	Description
	Next.js According to the official documentation, Next.js is a React framework for building full-stack web applications. It provides routing, rendering strategies, and built-in optimizations while automatically configuring lower-level tooling. This helped us build the frontend faster with a clear project structure and production-ready defaults [11].
	Node.js Node.js is officially presented as a free, open-source, cross-platform JavaScript runtime environment. It allows developers to create servers, web applications, command-line tools, and scripts with JavaScript. In this project, it provided the execution environment for backend services and API logic [12].
	Express The Express official site describes it as a fast, unopinionated, minimalist web framework for Node.js. It offers a flexible routing and middleware model with essential features for web and mobile APIs. We used Express to implement REST endpoints, middleware pipelines, and role-protected routes [13].
	PostgreSQL PostgreSQL is described by its official project as a powerful, open-source object-relational database system. It is recognized for reliability, data integrity, SQL support, and extensibility for complex workloads. In our platform, PostgreSQL stores users, work orders, stock records, notifications, and chat data [14].

Logo	Description
	Prisma ORM Prisma ORM is documented as a next-generation ORM for Node.js and TypeScript that provides type-safe database access. Its toolkit includes Prisma Client, Prisma Migrate, and Prisma Studio for querying, migrations, and data inspection. We used Prisma as the main data access layer to keep schema evolution and queries maintainable [15].
	Socket.IO Socket.IO is officially introduced as a library for low-latency, bi-directional, event-based communication. It supports transport fallback and includes features such as automatic reconnection and event acknowledgements. This was essential for real-time chat updates and immediate notification delivery in the application [16].
	Visual Studio Code The official VS Code website presents it as an open-source AI code editor and a world-class editor core used by millions of developers. It provides extension support, integrated terminal, debugging features, and rich language tooling. It was our primary development environment for frontend and backend implementation tasks [17].
	diagram.net diagrams.net (previously draw.io) is a graph drawing application written in JavaScript. It can be used to design and export many kinds of diagrams, including circuit diagrams, floor plans, flowcharts, infographics, mind maps, and UML designs. [18].
	LaTeX LaTeX is a high-quality typesetting system; it includes features designed for the production of technical and scientific documentation. LaTeX is the de facto standard for the communication and publication of scientific documents [19].

2.3 Global use case diagramm

the figure 2.1 illustrates the global use case diagram of the system, which represents the interactions between the actors and the main functionalities of the platform.

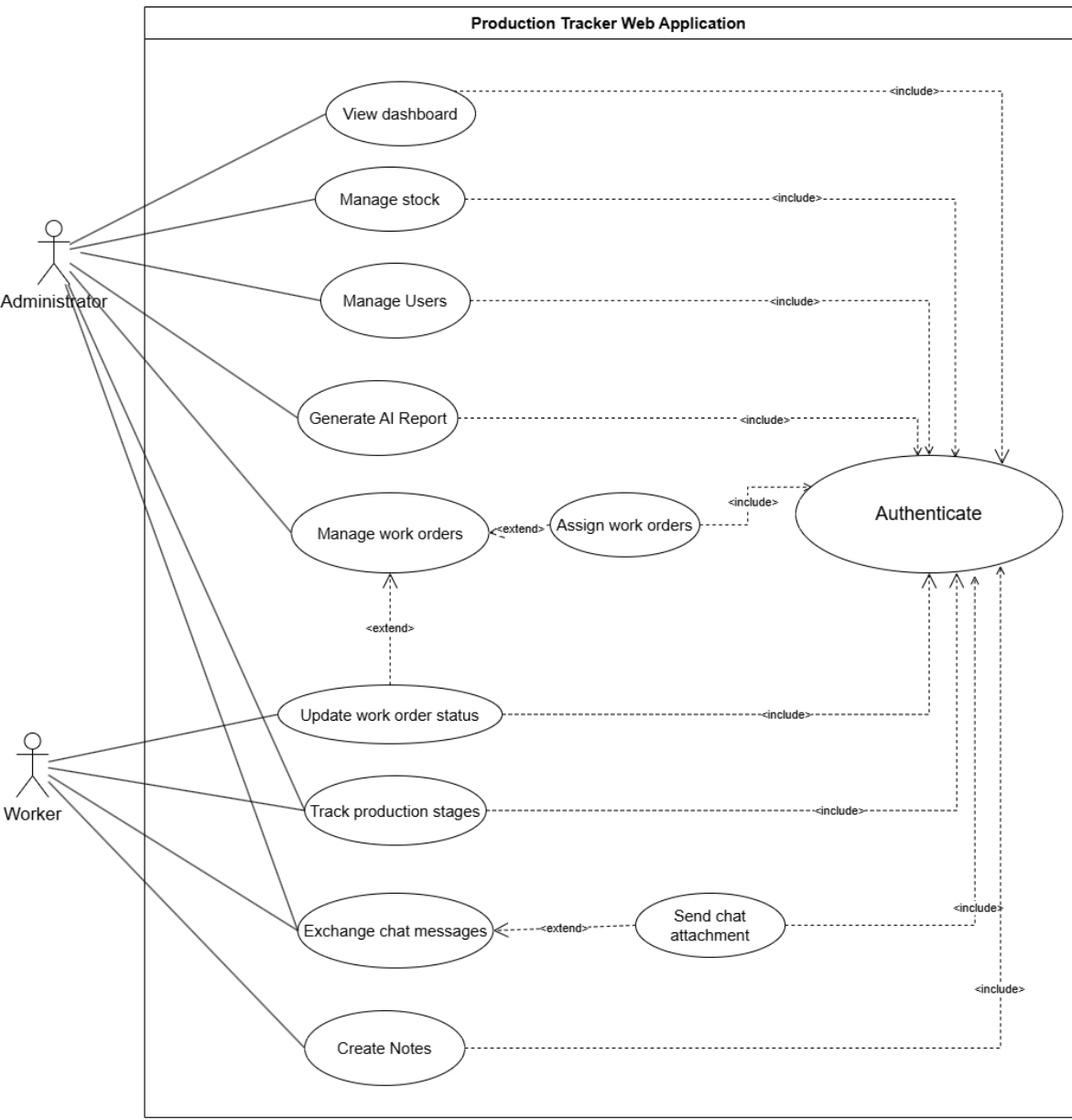


Figure 2.1: Use Case Diagram

2.4 Global class diagramm

the figure 2.2 illustrates the global class diagram of the system.

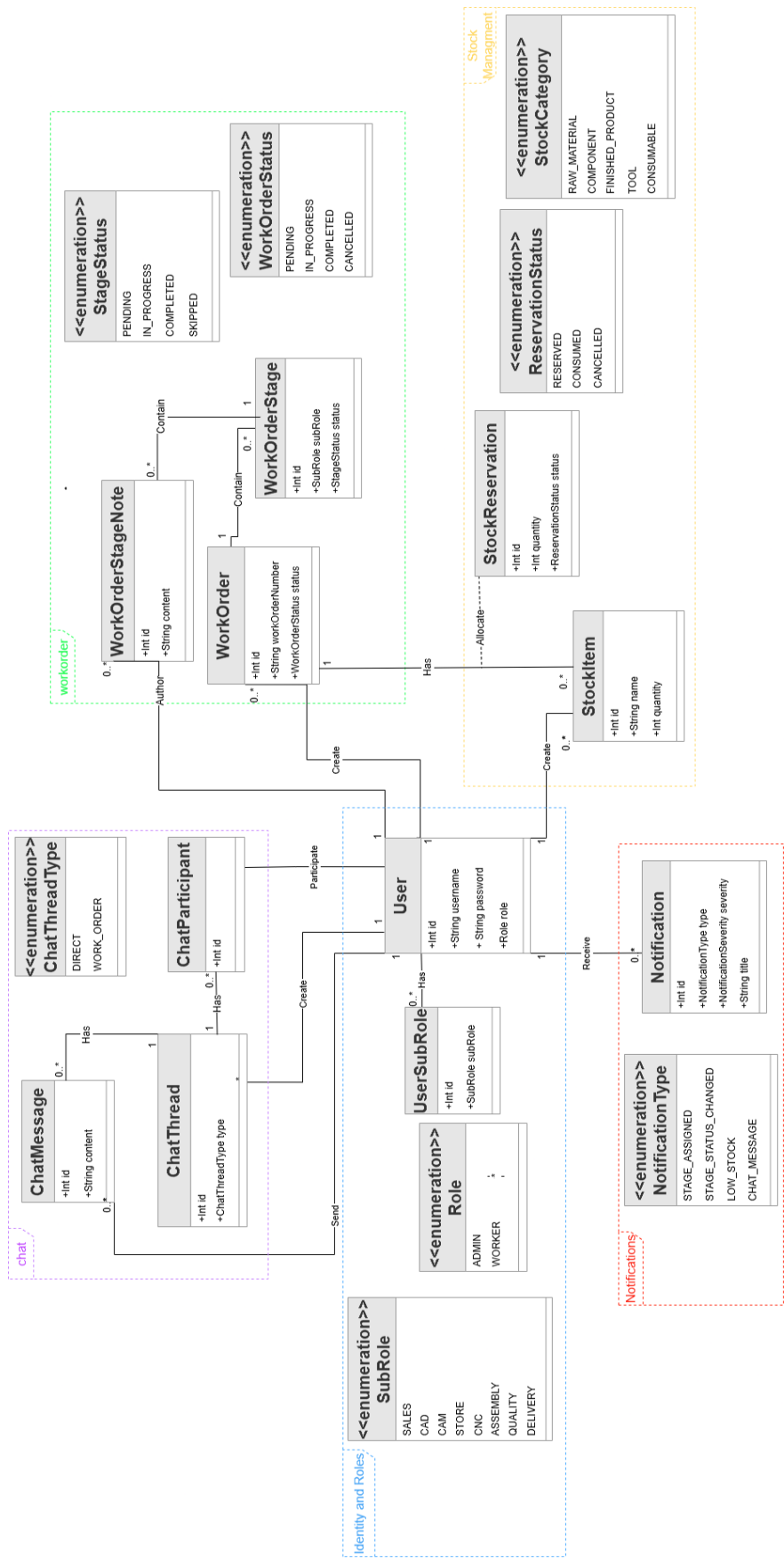


Figure 2.2: Global class diagram

2.5 Planning

2.5.1 Product backlog

in this section we present the product backlog, which is a prioritized list of features and requirements that will guide the development process. Each item in the backlog is defined as a user story, which describes a specific functionality from the perspective of the end user. The backlog is organized into sprints, with each sprint focusing on a set of related features that will be developed and delivered together. The Table 2.2 represents the global Product backlog.

Table 2.2: Product backlog

ID	Functionality	User Story	Actor
Sprint 1: Foundation and Access Control			
PB-01	User Authentication	As an administrator, I want to log in securely so that I can access protected management features.	Administrator
PB-02	Role-Based Access Control	As an administrator, I want role-based permissions so that users only access features related to their role.	Administrator
PB-03	User Management	As an administrator, I want to create, update, and deactivate user accounts so that I can manage the team effectively.	Administrator
PB-04	Session Security	As a user, I want secure session handling with token expiration so that my account remains protected.	Administrator / Worker
Sprint 2: Work Orders and Stage Tracking			
PB-05	Work Order Creation	As an administrator, I want to create work orders with required details so that production tasks are clearly defined.	Administrator
PB-06	Work Order Assignment	As an administrator, I want to assign work orders to workers so that each task has a responsible person.	Administrator
PB-07	Work Order Status Tracking	As a worker, I want to update the status of my assigned work orders so that progress is visible to the administrator in real time.	Worker
Sprint 3: Stock Management			
PB-08	Inventory Registration	As an administrator, I want to register stock items with unique identifiers so that materials can be tracked accurately.	Administrator
PB-09	Stock Reservation	As a worker, I want to reserve stock items for my tasks and mark them as consumed when used.	Worker

ID	Functionality	User Story	Actor
PB-10	Reservation Tracking	As an administrator, I want to monitor stock reservations so that inventory usage is auditable.	Administrator
Sprint 4: Notes and Notifications (Real-Time Chat included)			
PB-11	Stage Notes	As a worker, I want to add notes on work stages so that operational details are documented for traceability.	Worker
PB-12	Notification Center	As a user, I want to receive notifications for assignments, updates, and messages so that I do not miss important events.	Administrator / Worker
PB-13	Real-Time Chat	As a user, I want to chat with other users in real time so that I can discuss issues and coordinate.	Worker / Administrator
PB-14	File Sharing in Chat	As a user, I want to attach files in chat so that I can share production documents and evidence.	Administrator / Worker
Sprint 5: QR Codes, Analytics, and AI Reporting			
PB-15	QR Code Generation	As an administrator, I want generated QR codes for work orders so that identification is faster.	Administrator
PB-16	QR Code Scanning	As a user, I want to scan QR codes so that I can quickly search and access work orders.	Worker / Administrator
PB-17	AI Reporting Engine	As an administrator, I want AI-assisted reporting so that I can make faster data-driven decisions.	Administrator
PB-18	Analytics Dashboard	As an administrator, I want an analytical dashboard for production data so that I can monitor KPIs and trends.	Administrator

2.5.2 Sprint Planning

The internship lasted three and a half months, so the sprint planning was distributed accordingly.

The table 2.3 summarizes the sprint plan by priority and period.

Table 2.3: Sprint Plan

	Sprint	Priority	Period
Release 1	Foundation and Access Control	1	21 days
	Work Orders and Stage Tracking	2	21 days
	Stock Management	3	21 days
Release 2	Notes and Notifications	4	21 days
	Real-Time Chat, QR Codes, AI Reporting	5	21 days

2.6 Conclusion

This chapter provided a detailed analysis and specification of the requirements for the production tracking platform. Throughout this chapter, We identified the main actors involved in the system, defined functional and non-functional specifications to make sure the platform meets the company's needs, outlined the technical environment, In addition, we created several UML diagrams, including use case and class diagrams to provide a clear understanding of the system's behavior. Finally The detailed product backlog will serve as a roadmap for the development process and will guide the implementation of the platform. in the next chapter, we will start the implementation of the first release, which focuses on the development of the first three sprints.

Chapter 3

Release 1

3.1 Introduction

Release 1 presents the first three sprints of the Production Tracker project, focusing on building a solid foundation with authentication and access control, implementing the core production flow, and developing an inventory management system. This release provides the essential features needed to track work orders and manage stock efficiently.

3.2 Sprint 1: Foundation and Access Control

3.2.1 Sprint 1 Backlog

the table 3.1 summarizes the user stories and tasks completed during Sprint 1.

Table 3.1: Sprint 1 Backlog

ID	User Story	Tasks
PB-01	As an administrator, I want to log in securely so that I can access protected features.	Setup authentication module; Implement login/logout; Add password hashing
PB-02	As an administrator, I want role-based permissions so that users access only appropriate features.	Define role models; Implement permission checks; Create role management UI
PB-03	As an administrator, I want to create and manage user accounts so that the team can access the system.	Build user CRUD operations; Add user validation; Create user management interface
PB-04	As a user, I want secure session handling so that my account remains protected.	Implement JWT tokens; Add session expiration; Create token refresh mechanism

3.2.2 Sprint 1 use case diagram

the figure 3.1 illustrates the main use cases implemented in Sprint 1, focusing on user authentication and management features. It shows the interactions between the administrator and the system for logging in, and managing user accounts.

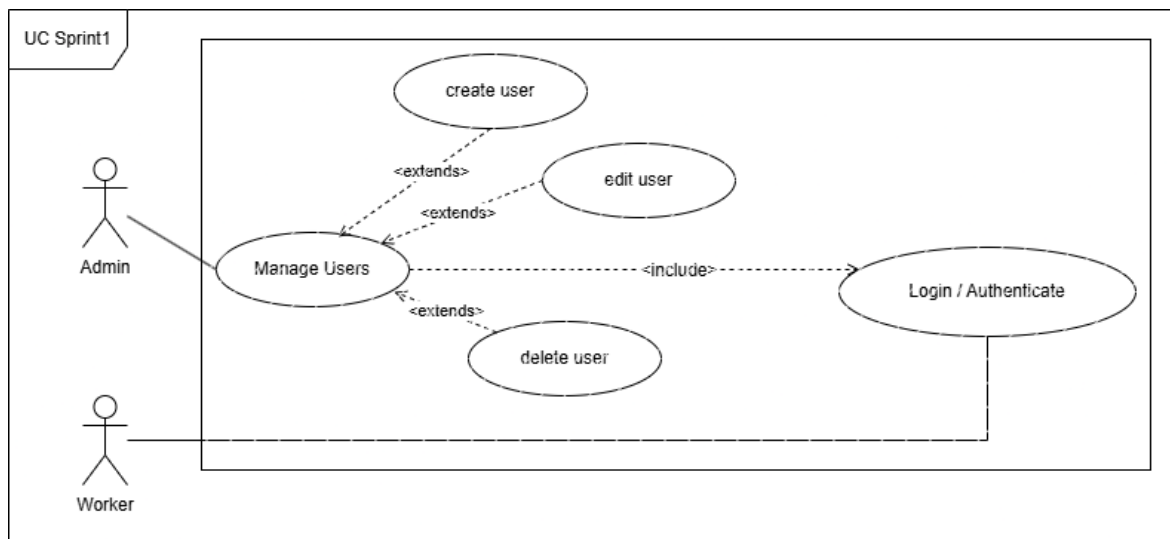


Figure 3.1: Sprint 1 Use Case Diagram

3.2.3 User Login and Protected Resource Access — JWT Sequence Diagram

JSON Web Token (JWT) is a proposed Internet standard for creating data with optional signature and/or optional encryption whose payload holds JSON that asserts some number of claims. The tokens are signed either using a private secret or a public/private key. [20]

the figure 3.2 illustrates the sequence of interactions during the authentication process using JWT tokens.

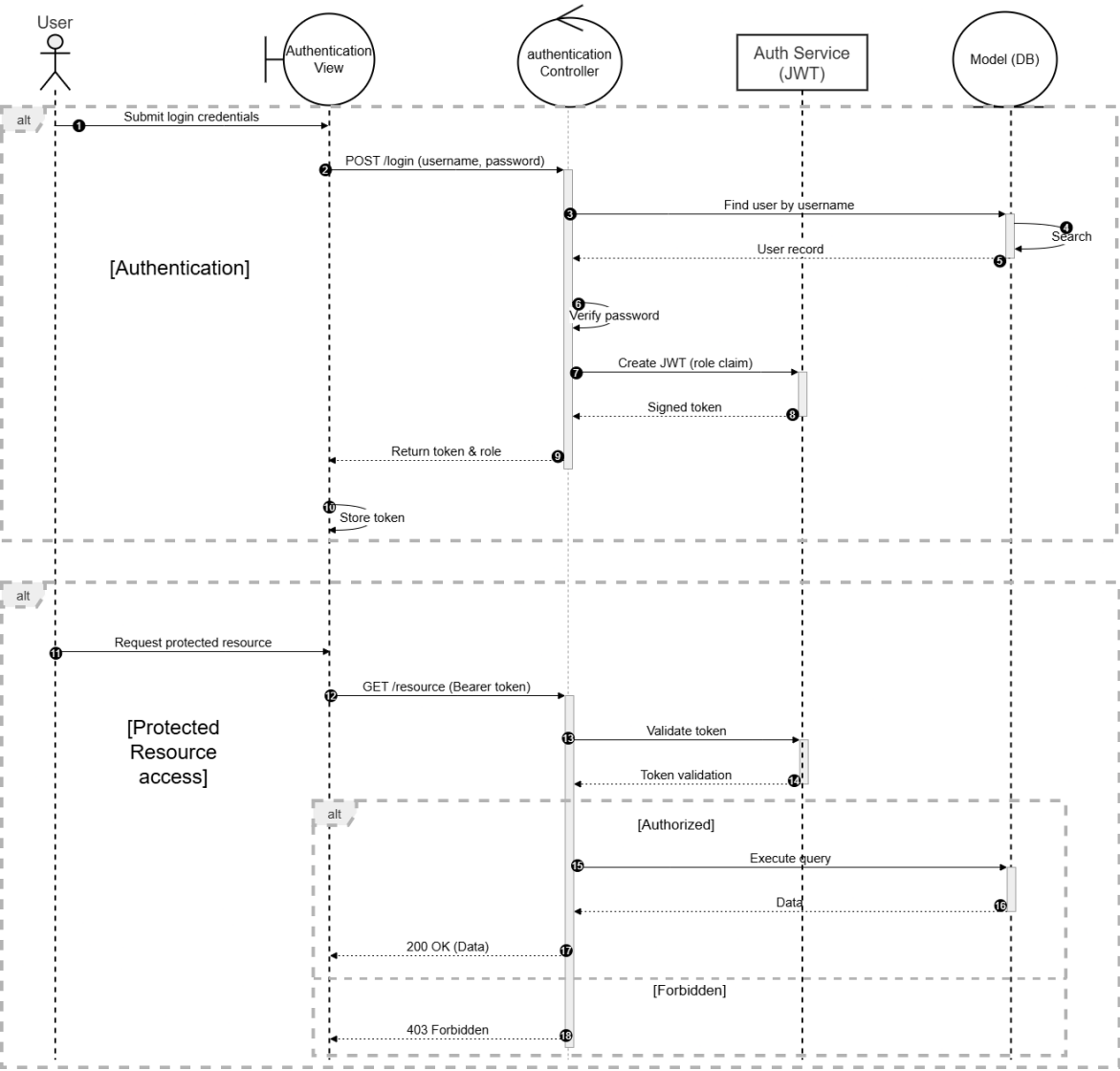


Figure 3.2: Authentication Sequence Diagram

3.2.4 Sprint 1: Application Interface Overview

The figure 3.3 show the login interface that represents the gate to access the platform. While the figures 3.4 and 3.5 show the user management interface that allows administrators to create and manage user accounts, including a modal for creating new users with role assignments and validation features.

ID	User Story	Tasks
PB-06	As an administrator, I want to assign work orders to workers so that each task has a responsible person.	Create assignment workflow; Build assignment UI; Add notification trigger
PB-07	As a worker, I want to update my work order status so that progress is visible in real time.	Build status update endpoints; Create status UI; Add real-time updates

3.3.2 Sprint 2 use case diagram

the figure 3.6 illustrates the main use cases implemented in Sprint 2, focusing on work order management and stage tracking features. It shows the interactions between the administrator and workers with the system for creating, assigning, and updating work orders.

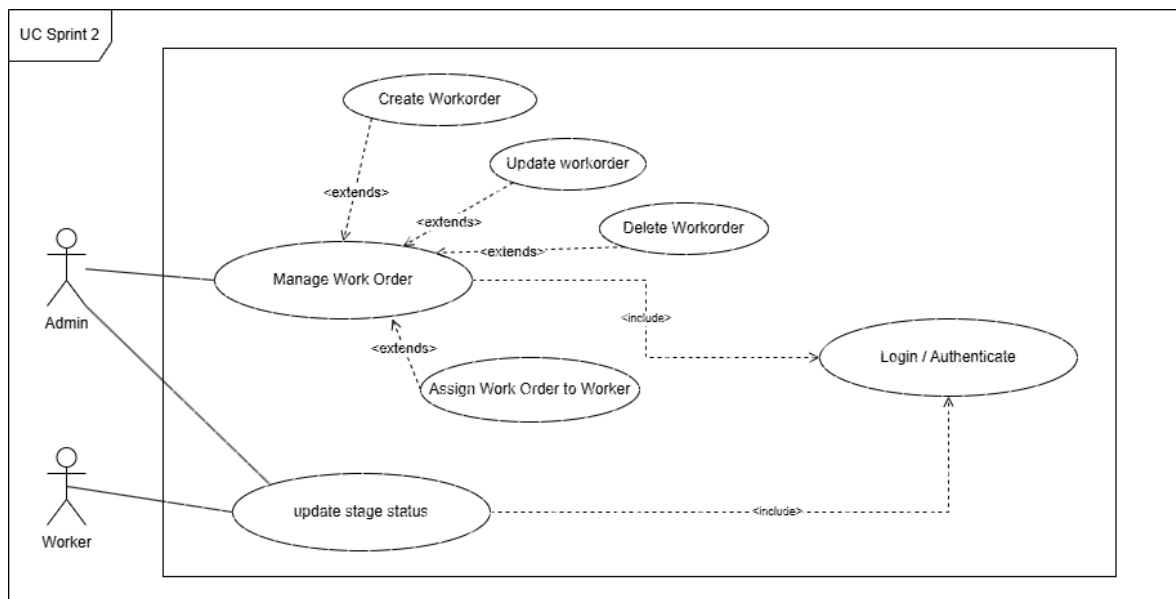


Figure 3.6: Sprint 2 Use Case Diagram

3.3.3 Sequence Diagram for Work Order Progress Update

the figure 3.7 illustrates the sequence of interactions during the work order progress update process, showing how a worker updates the status of a work order and how the system processes this update to reflect changes in the production workflow.

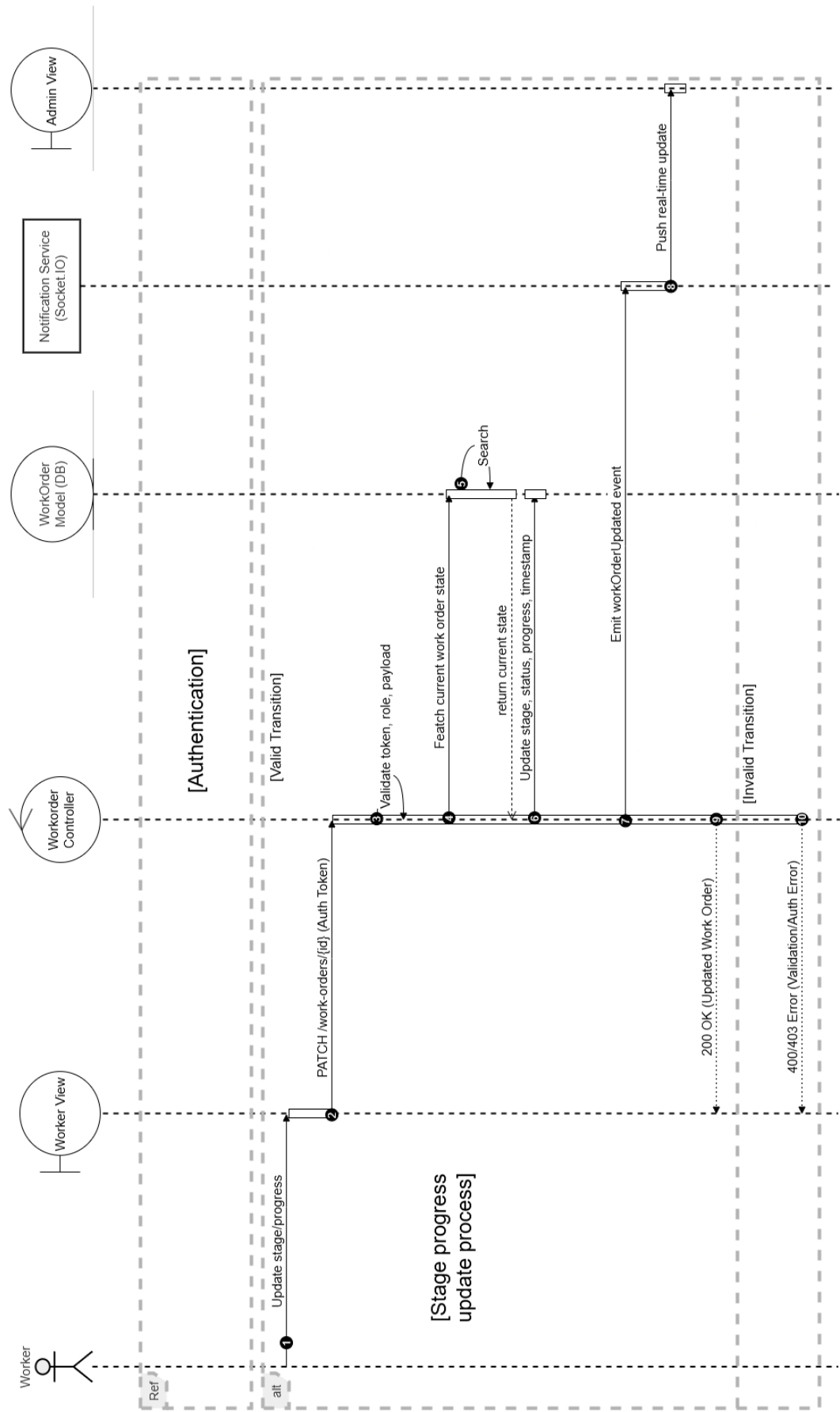


Figure 3.7: Work Order Progress Update Sequence Diagram

3.3.4 Sprint 2: Application Interface Overview

the figures 3.8, 3.9, and 3.10 show the interfaces related to work order management, production stages administration, and worker task views, respectively. These interfaces allow administrators to manage work orders and production stages while providing workers with a clear view of their assigned tasks and progress.

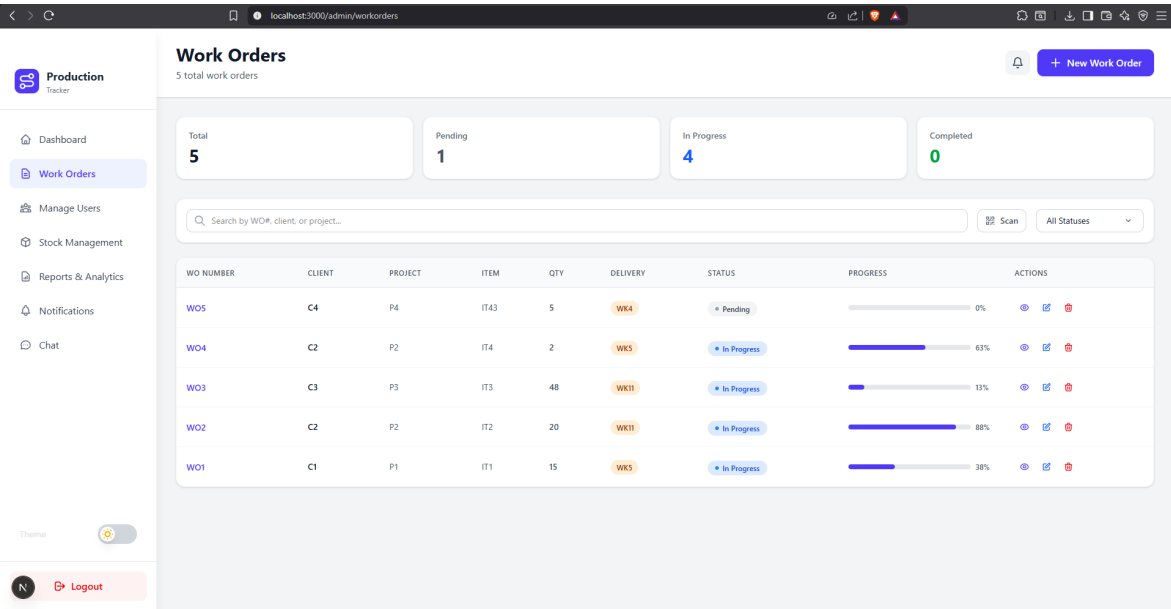


Figure 3.8: Work Order Management Interface

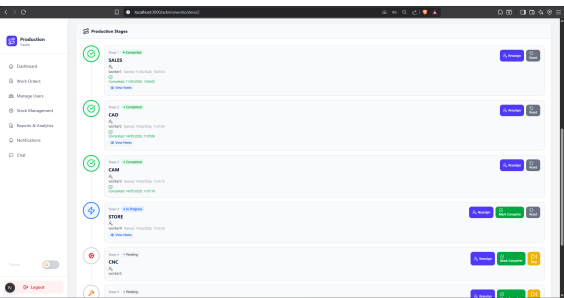


Figure 3.9: Production Stages Admin View

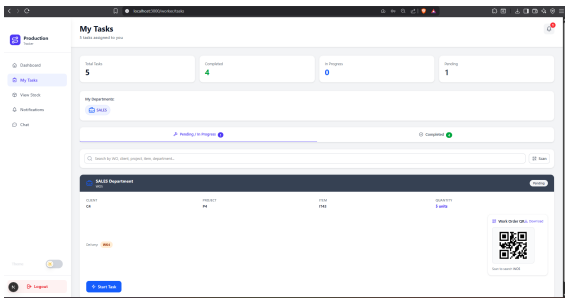


Figure 3.10: Worker Tasks View

3.4 Sprint 3: Stock Management

3.4.1 Sprint Backlog

the table 3.3 summarizes the user stories and tasks completed during Sprint 3.

ID	User Story	Tasks
PB-08	As an admin, I want to create, edit, and delete stock items so that inventory stays accurate.	Design inventory schema; Build stock registration UI; create Stock CRUD works
PB-09	As a worker, I want to reserve stock items for my tasks and mark them as consumed when used.	create reservation logic; build reservation interface;
PB-10	As an administrator, I want to monitor stock reservations	create reservation tracking functionality; Build reservation tracking UI;

Table 3.3: Sprint 3 Backlog

3.4.2 Sprint 3 use case diagram

the figure 3.11 illustrates the main use cases implemented in Sprint 3, focusing on stock management features. It shows the interactions between administrators and workers with the system for managing stock items, reserving inventory, and monitoring reservations.

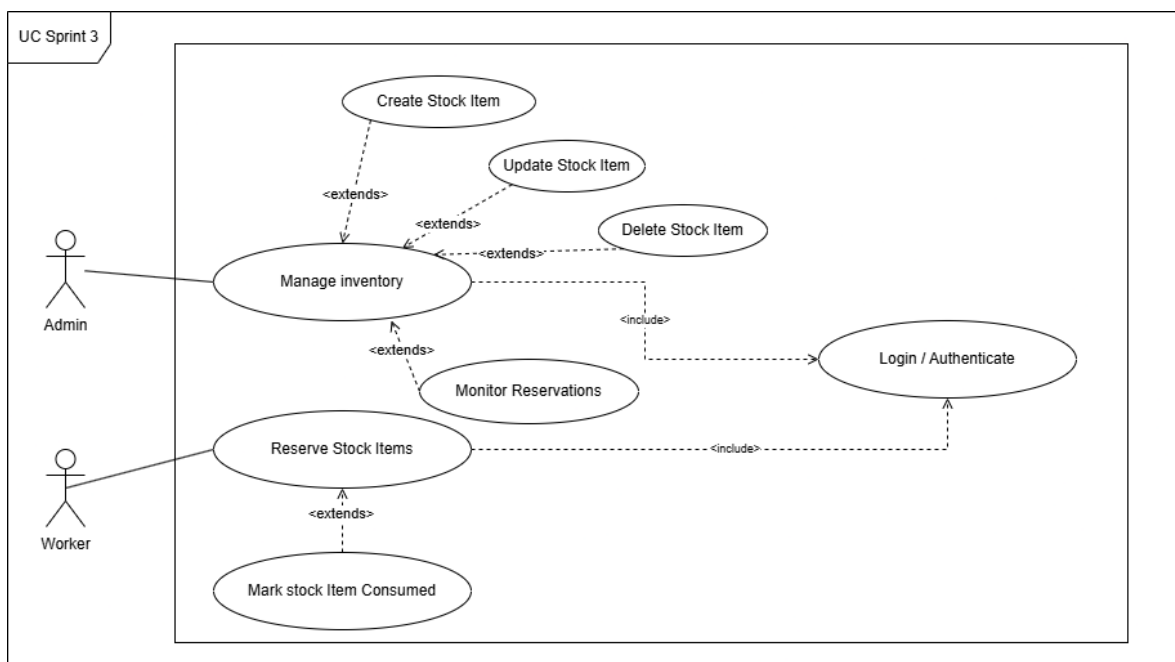


Figure 3.11: Sprint 3 Use Case Diagram

3.4.3 Sprint 3: Application Interface Overview

the figures 3.12 and 3.13 show the inventory management interfaces for administrators, allowing them to manage stock items and monitor reservations.

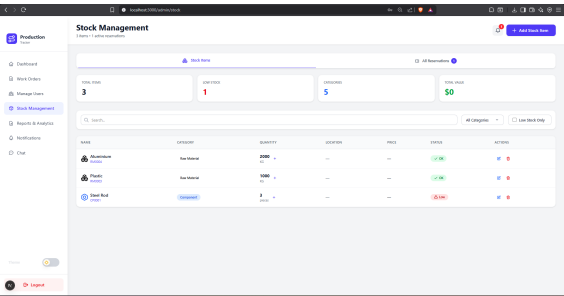


Figure 3.12: Inventory Admin View

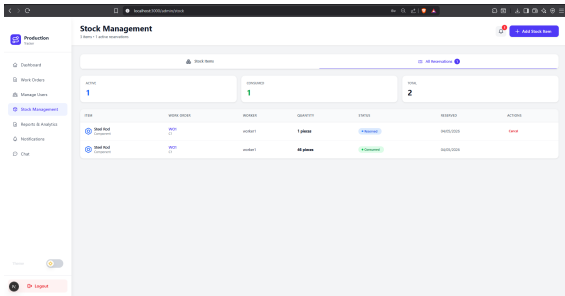


Figure 3.13: Inventory Reservations Admin View

The figures 3.14 and 3.15 show the inventory interfaces for workers, enabling them to view available stock and manage their reservations effectively.

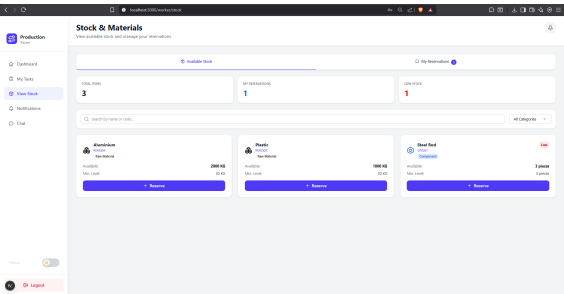


Figure 3.14: Inventory Worker View

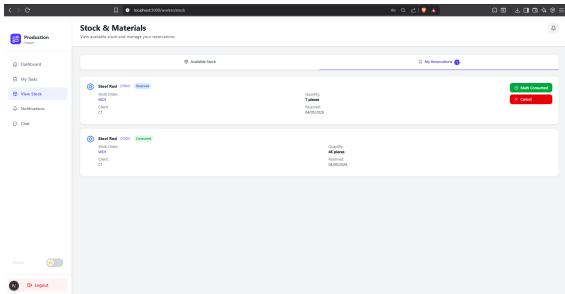


Figure 3.15: Inventory Reservations Worker View

3.5 Conclusion

The first release completed the core features that formed the foundation of the Production Tracker software, by providing security through the implementation of authentication processes, building the core functionality that formed the production flow process, and finally, implementing inventory management. These three sprints lasted 21 days each, giving a total of 63 days, after which the second release will be covered in the next chapter.

Chapter 4

Release 2

4.1 Introduction

Release 2 encompasses the final two sprints of the Production Tracker project, focusing on real-time communication through integrated chat systems, quick identification with QR code technology, intelligent reporting with AI assistance, and comprehensive notification management. This release delivers advanced features that enhance user collaboration, operational visibility, and decision-making capabilities.

4.2 Sprint 4: Notes, Real-Time Chat, and Notifications

4.2.1 Sprint Backlog

the table 4.1 summarizes the user stories and tasks completed during Sprint 4:

Table 4.1: Sprint 4 Backlog

ID	User Story	Tasks
PB-11	As a worker, I want to add notes on work stages so that operational details are documented.	Create notes logic; Build notes UI; Add note timestamp and author tracking
PB-12	As a user, I want to chat with other users in real time so that I can discuss issues.	Setup real-time communication; Build chat interface; Implement message history
PB-13	As a user, I want to attach files in chat so that I can share documents and evidence.	Build file upload handler; Add file attachment UI
PB-14	As a user, I want to receive notifications so that I do not miss important events.	Implement notification service; Build notification UI; Add notification management

4.2.2 Sprint 4 Use Case Diagram

the figure 4.1 illustrates the main use cases implemented in Sprint 4, focusing on real-time communication and notification features. It shows the interactions between users for adding notes, engaging in chat conversations, attaching files, and managing notifications within the system.

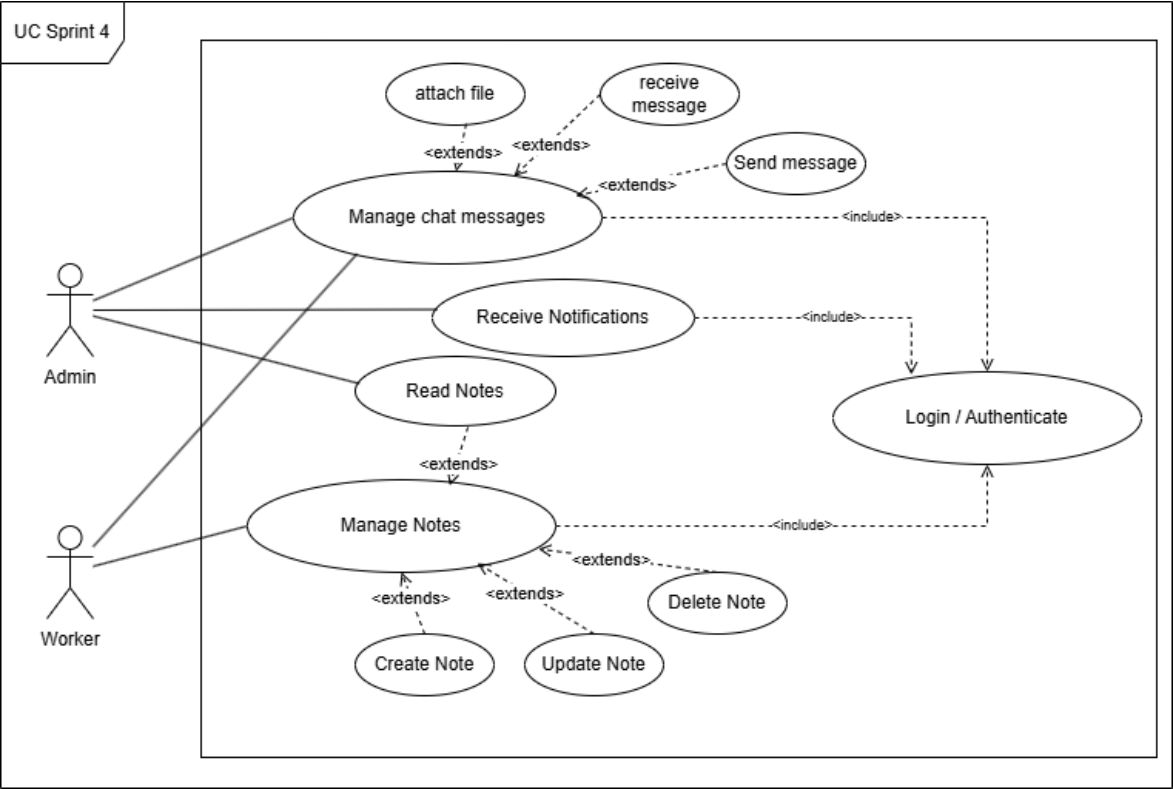


Figure 4.1: Sprint 4 Use Case Diagram

4.2.3 Sequence Diagram for Real-Time Messaging

the figure 4.2 illustrates the sequence of interactions that occur during a real-time chat session between two users. It shows how messages are sent, received, and persisted in the system, highlighting the use of Socket.IO for real-time communication and the integration with the database for message storage.

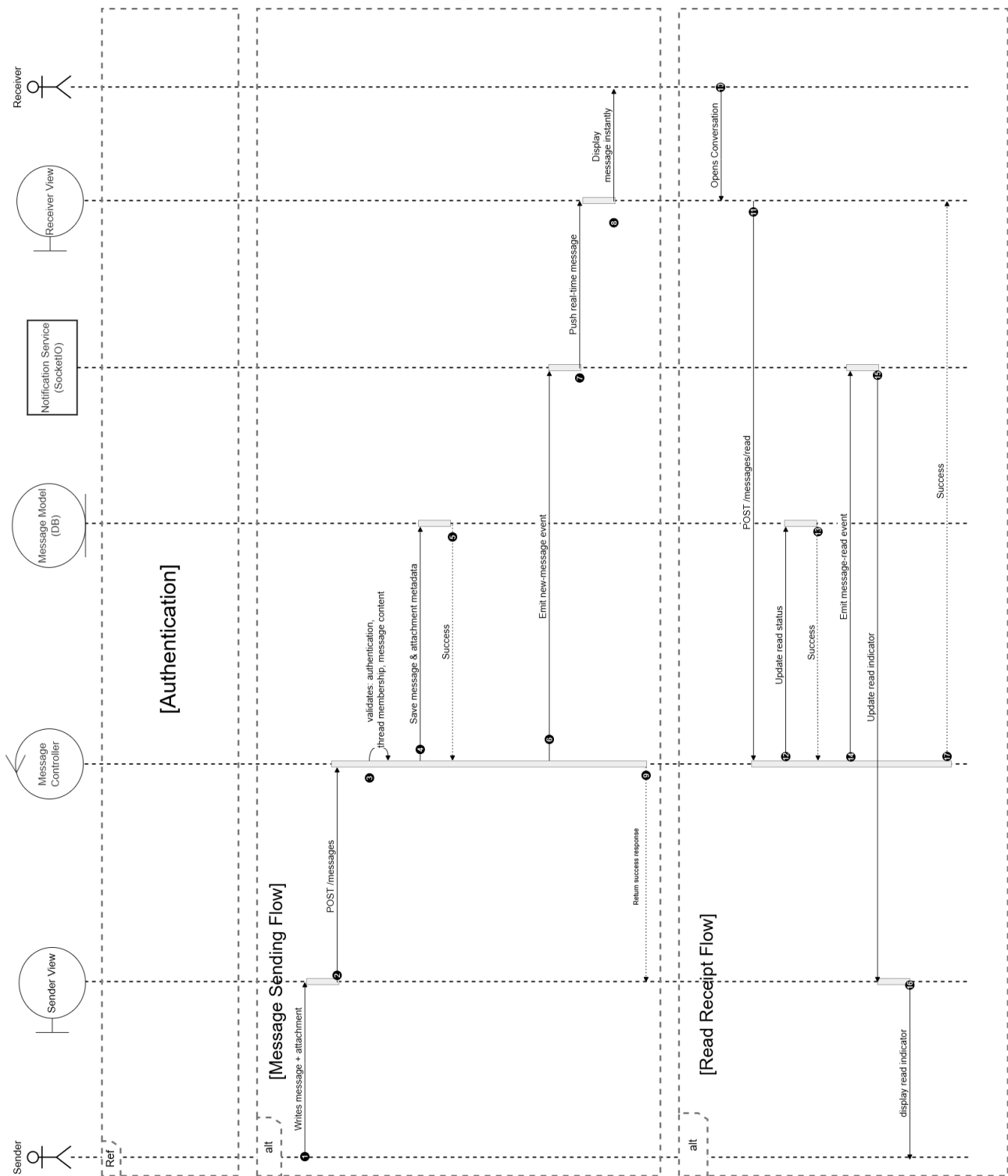


Figure 4.2: Real-Time Chat Message Flow Sequence Diagram

4.2.4 Sprint 4: Application Interface Overview

the figures 4.3, 4.4 illustrate the note management interfaces for workers and administrators, respectively. The worker view allows users to add and view notes related to their tasks, while the admin view provides a comprehensive history of all notes for monitoring and analysis.

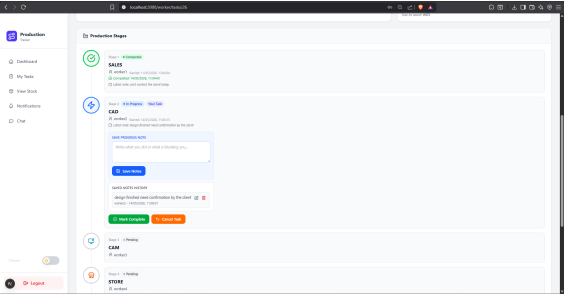


Figure 4.3: Note Management Worker View

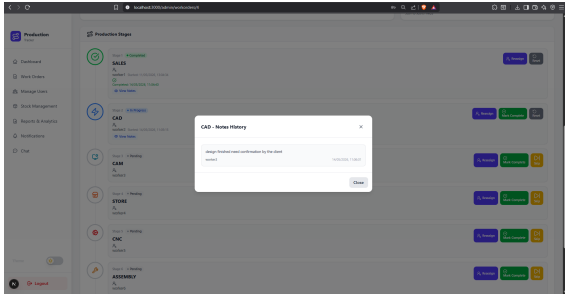


Figure 4.4: Notes History Admin View

the figures 4.5 and 4.6 provide an overview of the real-time chat messaging interfaces.

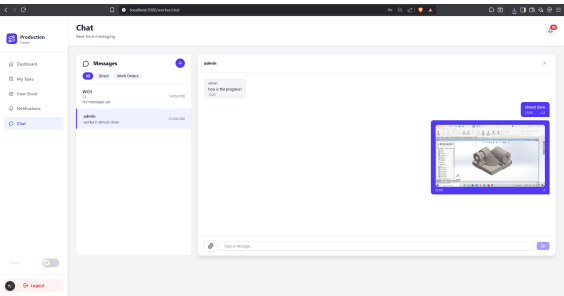


Figure 4.5: Chat Messaging Interface Worker View

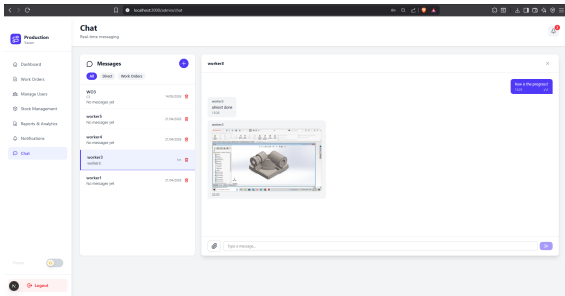


Figure 4.6: Chat Messaging Interface Admin View

the figure 4.7 illustrates the notification center interface for managing alerts and notifications within the system.

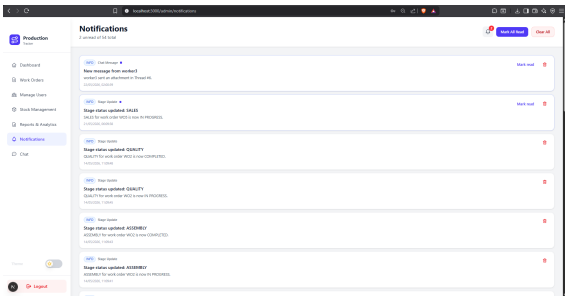


Figure 4.7: Notification Center

4.3 Sprint 5: QR Codes, Analytics, and AI Reporting

4.3.1 Sprint Backlog

the table 4.2 summarizes the user stories and tasks completed during Sprint 5:

Table 4.2: Sprint 5 Backlog

ID	User Story	Tasks
PB-15	As an administrator, I want generated QR codes for work orders so that identification is faster.	Implement QR code library; Build generation API; Add QR display
PB-16	As a user, I want to scan QR codes so that I can quickly search and access work orders .	Integrate QR scanner; Build scan interface; Add scan result handler
PB-17	As an administrator, I want AI-assisted reporting so that I can make faster decisions.	Setup reporting engine; Integrate AI analysis; Build dashboard UI
PB-18	As an administrator, I want a analytical dashboard for the production data.	Build analytics UI; Integrate data visualization;

4.3.2 Sprint 5 Use Case Diagram

the figure 4.8 illustrates the main use cases implemented in Sprint 5:

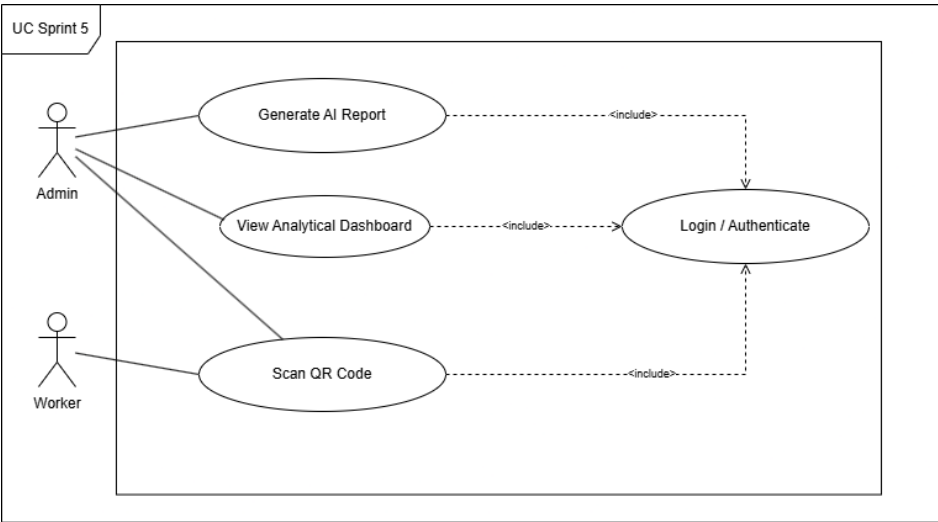


Figure 4.8: Sprint 5 Use Case Diagram

4.3.3 Sequence Diagram for AI-Based Report Generation

the figure 4.9 illustrates the sequence of interactions that occur during the AI-based report generation process. It shows how data is collected, processed, and analyzed using AI to generate insightful reports for administrators.

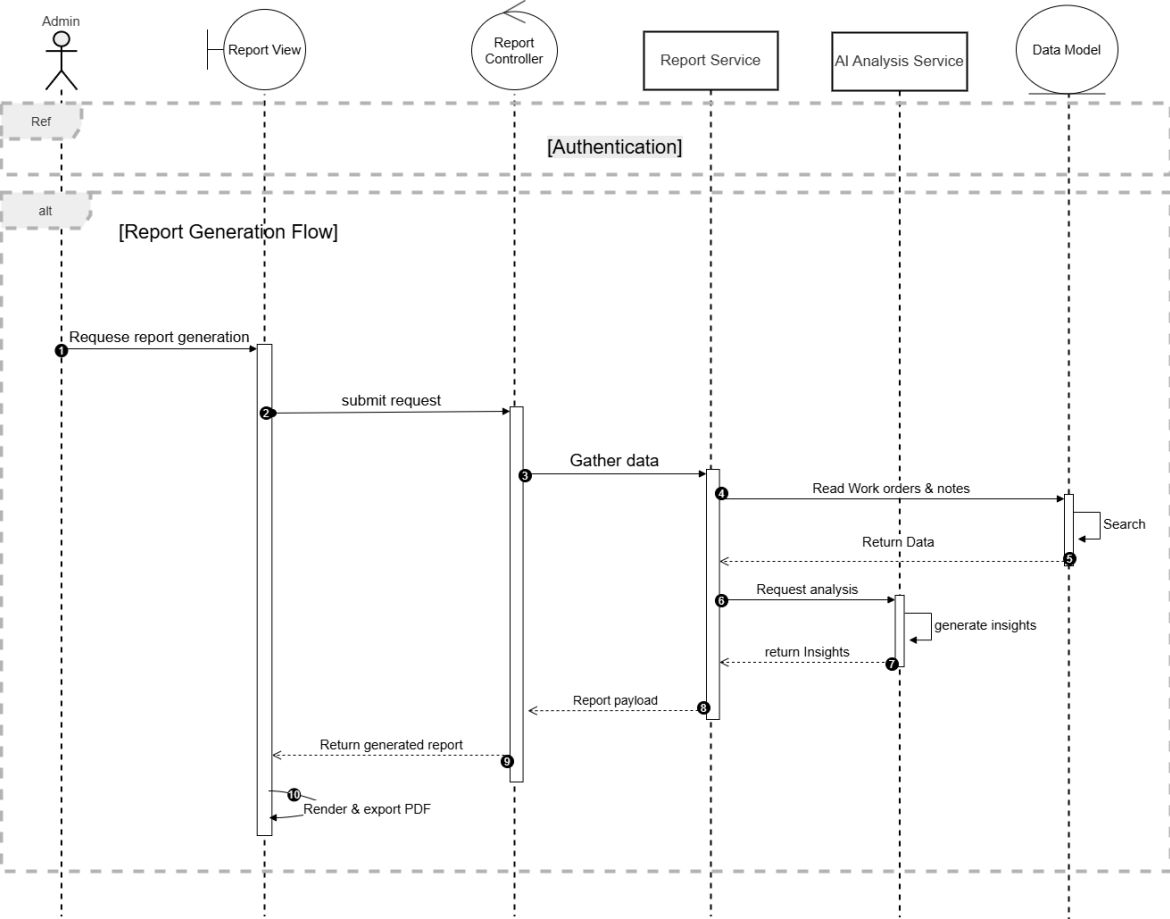


Figure 4.9: Sequence Diagram for AI-Based Report Generation

4.3.4 Sprint 5: Application Interface Overview

the figures 4.10 and 4.11 illustrate the interfaces for work order details with QR code display and the QR code scanner, respectively. The work order details interface allows users to view comprehensive information about a work order along with a generated QR code for quick access, while the QR code scanner interface enables users to scan codes and retrieve associated work order information efficiently.

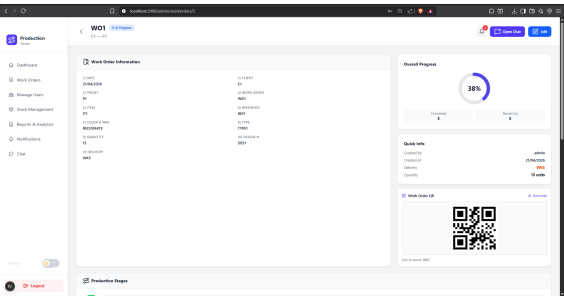


Figure 4.10: Work Order Details Interface with QR Code

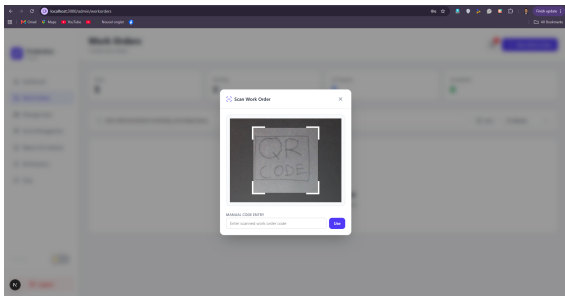


Figure 4.11: QR Code Scanner Interface

the figures 4.12 and 4.13 provide an overview of the AI report interface and the analytics dashboard, respectively. The AI report interface allows administrators to generate and view AI-assisted reports, while the analytics dashboard provides visualizations and insights based on production data for informed decision-making.

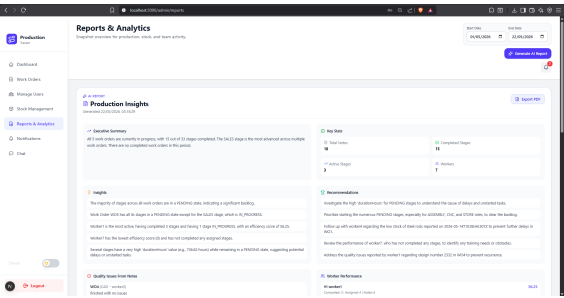


Figure 4.12: AI Report Interface

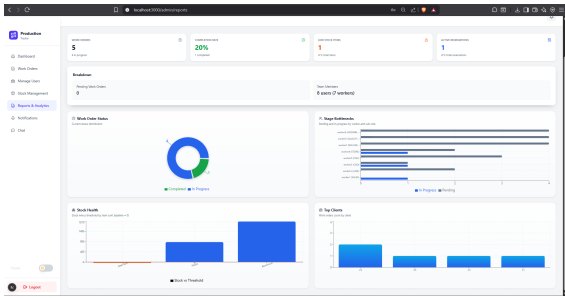


Figure 4.13: Analytics Dashboard

4.4 Conclusion

Release 2 proved to be successful since it provided advanced functionalities that converted the Production Tracker into a fully-featured collaboration tool. The two sprints in Release 2 took 21 days each (summing up to 42 days), incorporating real-time communication features, effective identifying methods, and intelligent reporting. Combining Release 2 with Release 1 made up the 5-sprint process taking a total of 105 days to develop the application.

General Conclusion

This end-of-studies project was driven by a real operational need at KTS: replacing a paper-based production tracking process with an integrated digital platform. Over the course of this report, we progressed from problem analysis to technical design and then to the practical implementation of a web application designed to better structure daily production activities and improve coordination among stakeholders.

The developed solution, **Production Tracker**, addresses the core business workflows identified during the analysis phase. It provides role-based authentication, work order management across the main production stages, stock monitoring and reservation, real-time notifications, and instant communication through chat. By bringing these functions together in a single system, the platform reduces information dispersion, limits manual errors, and improves the traceability of production operations.

From a technical perspective, the chosen stack (Next.js, Node.js/Express, PostgreSQL, Prisma, and Socket.IO) proved well aligned with the project objectives. It made it possible to build a modular architecture, ensure a clear separation of responsibilities between frontend and backend, and develop both transactional and real-time features efficiently. The resulting system offers a solid foundation for maintainability and future evolution.

Beyond the technical implementation, this work brings clear organizational value to the host company. Administrators now have better visibility over production progress, workers receive clearer task guidance, and communication delays are reduced through shared real-time information. In this way, the project contributes directly to KTS's digital transformation effort and supports more reliable day-to-day decision making.

Like any first complete version, the current platform can still be improved. Future work may include advanced analytics dashboards, richer KPI reporting, tighter integration with ERP systems, mobile-first access for shop-floor usage, and additional automation for planning and forecasting. These extensions would strengthen strategic monitoring while preserving the operational gains already achieved.

In conclusion, this internship project turned a real industrial problem into a functional and scalable software solution. It also marked an important personal and professional milestone, bringing together software engineering practices, system design, and iterative delivery in a real business context.

References

- [1] **Manufacturing Execution System (MES) Definition** — Wikipedia. https://en.wikipedia.org/wiki/Manufacturing_execution_system
- [2] **KTS (Kabelbaum Test Systems) introduction** . <https://www.ktsistem.com/kurumsal.php?dl=en>
- [3] **KTS (Kabelbaum Test Systems) Logo** . <https://ktsistem.com/kurumsal.php?dl=en>
- [4] **SMOFT ERP Official Website**. <https://www.smoft.io>
- [5] **SMOFT ERP LOGO**. <https://www.facebook.com/smofterp/>
- [6] **SAP Digital Manufacturing**. [SAP Digital Manufacturing official page](#)
- [7] **SAP Digital Manufacturing Logo**. [SAP Digital Manufacturing logo](#)
- [8] **SAP Digital Manufacturing Dashboard**. [SAP Digital Manufacturing official page](#)
- [9] **The Scrum Guide** — Scrum Guides. <https://scrumguides.org/>
- [10] **The Scrum Process** <https://www.pm-partners.com.au/insights/the-agile-journey-a-s>
- [11] **Next.js Documentation**. <https://nextjs.org/docs>
- [12] **Node.js Official Website**. <https://nodejs.org/en>
- [13] **Express Official Website**. <https://expressjs.com/>
- [14] **PostgreSQL About Page**. <https://www.postgresql.org/about/>
- [15] **Prisma ORM Documentation**. <https://www.prisma.io/docs/orm/overview/introduction/what-is-prisma>
- [16] **Socket.IO Documentation (v4)**. <https://socket.io/docs/v4/>
- [17] **Visual Studio Code Official Website**. <https://code.visualstudio.com/>
- [18] **diagrams.net Documentation - wikipedia**. <https://en.wikipedia.org/wiki/Diagrams.net>
- [19] **LaTeX Official Website**. <https://www.latex-project.org/>
- [20] **JSON Web Token (JWT)**. https://en.wikipedia.org/wiki/JSON_Web_Token